

PF COM HASKELL

Pedro Henrique Ribeiro Dias

Inatel

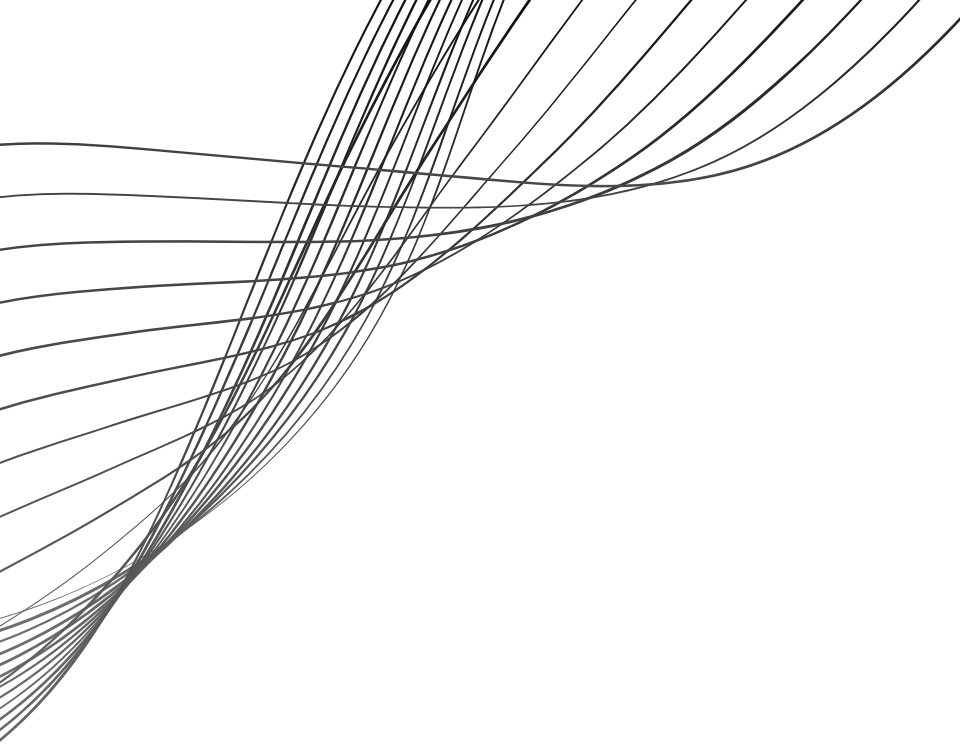
INTRODUÇÃO

Haskell: A Linguagem que Pensa em Funções

Diferente de tudo que foi visto até aqui, Haskell não organiza o código em objetos nem em instruções sequenciais – ele organiza em transformações. Lançado em 1990 e construído sobre décadas de pesquisa em matemática, Haskell é a linguagem que mais fielmente representa o paradigma funcional puro.

O que torna o Haskell único?

- **Imutabilidade Total:** Em Haskell, variáveis não variam. Um valor definido nunca muda – o que elimina uma categoria inteira de bugs causados por estado compartilhado.
- **Funções Puras:** Uma função sempre retorna o mesmo resultado para os mesmos argumentos, sem efeitos colaterais. O código se torna previsível e fácil de testar.
- **Tipagem Estática e Inferida:** Haskell tem um dos sistemas de tipos mais rigorosos que existem – mas raramente você precisa escrevê-los, o compilador deduz sozinho.



INTRODUÇÃO

"Em Haskell, você não diz ao computador o que fazer – você descreve o que as coisas são. É uma mudança de mentalidade, não só de sintaxe."

PILARES DA PF

Funções Puras: a saída depende apenas da entrada. A mesma chamada com os mesmos argumentos sempre retorna o mesmo resultado – sem modificar nada fora dela.

Imutabilidade: uma vez que um valor é definido, ele não muda. Isso elimina efeitos colaterais e torna o fluxo de dados muito mais fácil de acompanhar.

Expressões vs. Comandos: na PF não existem instruções que alteram estado – tudo é uma expressão que retorna um valor. Você descreve o que é, não o que fazer.

Composição de Funções: funções pequenas e simples são combinadas para formar comportamentos mais complexos, como blocos de montar.

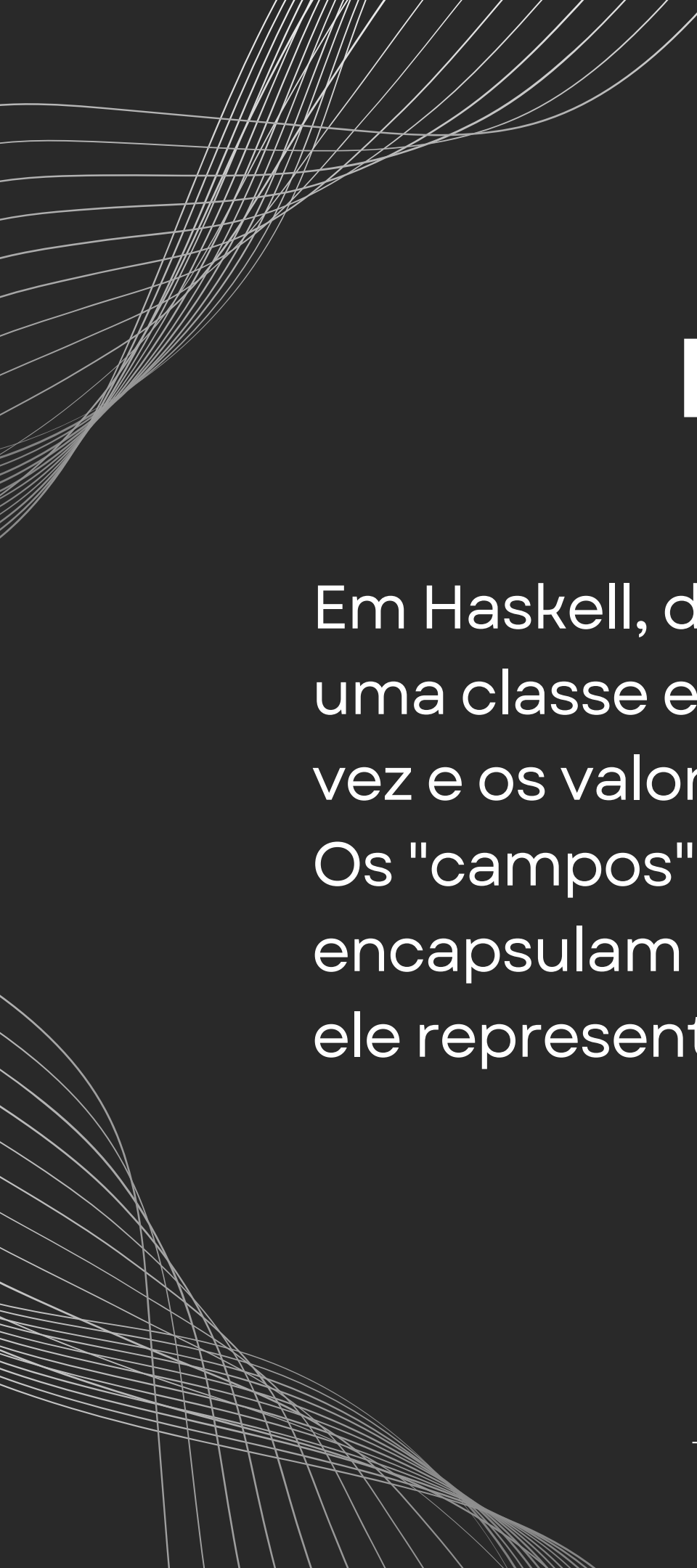
POR QUE HASKELL?

- Funcional Puro: todas as funções são puras por padrão – não existe forma de burlar isso acidentalmente, o que força boas práticas desde o início.
- Fortemente Tipada: todos os tipos são verificados em tempo de compilação pelo GHC. Se o programa compila, boa parte dos erros já foi eliminada antes de rodar.
- Inferência de Tipos: apesar da tipagem rígida, você raramente precisa declarar os tipos manualmente – o compilador deduz sozinho na maioria dos casos.

IMUTABILIDADE E FUNÇÕES PURAS

Imutabilidade: em Haskell, $x = 5$ não é uma atribuição – é uma definição. Não existe $x = x + 1$ porque x não varia, ele é aquele valor. Para "atualizar" algo, você cria um novo valor a partir do anterior.

Função Pura: uma função depende apenas dos seus argumentos e nada mais. Sem acesso a variáveis externas, sem modificar estado, sem surpresas – a mesma entrada sempre produz a mesma saída, garantido.



TIPOS PERSONALIZADOS DATA

Em Haskell, data é a forma de criar suas próprias estruturas, parecido com uma classe em OO – mas imutável por natureza. Você define o tipo uma vez e os valores criados a partir dele nunca mudam.

Os "campos" do tipo são chamados de Construtores de Dados e encapsulam as informações daquela estrutura. Uma vez criado um valor, ele representa exatamente aquilo – sem alterações posteriores.

LISTAS EM HASKELL

Listas são a estrutura de coleção principal do Haskell e só aceitam elementos do mesmo tipo. Em vez de acessar elementos por índice, usamos Pattern Matching – uma forma de descrever o formato esperado da lista:

- `[]` casa com uma lista vazia
- `(x:xs)` separa o primeiro elemento `x` do restante `xs`
- `(x:_)` pega só o primeiro e ignora o resto com `_`

É uma mudança de mentalidade: em vez de navegar pela lista, você descreve o que espera encontrar.

FUNÇÕES DE ALTA ORDEM

Funções de Alta Ordem são funções que recebem outras funções como argumento – o que permite criar comportamentos genéricos e reutilizáveis.

- map: aplica uma função a cada elemento de uma lista e retorna uma nova lista com os resultados. O original nunca é modificado.
- filter: percorre a lista e retorna só os elementos que satisfazem uma condição.
- sum: reduz uma lista inteira a um único valor somando todos os elementos.

CONDIÇÕES FUNCIONAIS

GUARDS

Guards são a forma do Haskell lidar com condições. Funcionam como um if/else, mas de forma muito mais legível – cada condição fica em uma linha separada com |, e o programa executa o primeiro caso verdadeiro que encontrar.

Além de mais limpo visualmente, o uso de guards reforça o estilo declarativo do Haskell: você descreve em que condição cada resultado se aplica, não como chegar até ele.

IO E A FUNÇÃO MAIN

Haskell separa de forma estrita o código puro do código que interage com o mundo externo. Qualquer operação de entrada e saída vive dentro do tipo IO – e isso não é opcional, o compilador garante essa separação.

- `main`: é o ponto de entrada do programa e o único lugar onde ações IO podem ser executadas em sequência
- `putStrLn`: exibe uma string com quebra de linha
- `putStr`: exibe sem quebrar a linha
- `print`: exibe qualquer valor que possa ser convertido em texto

EXERCÍCIO 1

Exercício 1 – Café Leblanc

No Café Leblanc, Sojiro anota cada pedido feito durante o dia. Cada bebida tem um nome, um tipo e um preço. Ao final, ele precisa saber o valor total do pedido.

- Defina o tipo `Bebida` com os campos `nome`, `tipo` e `preco`.
- Crie um tipo `StatusPedido` com os construtores `Aberto`, `Entregue` e `Cancelado`.
- Crie o tipo `Pedido` com uma lista de bebidas e um `StatusPedido`.
- Implemente `valorTotalPedido :: Pedido -> Double` que soma os preços das bebidas. Use guards para retornar 0.0 se o pedido estiver `Cancelado`, e o total simples nos demais casos.
- Implemente `primeiraBebida :: Pedido -> String` usando pattern matching na lista de bebidas – retorne o nome da primeira ou uma mensagem caso a lista esteja vazia.
- Na main, crie dois pedidos com bebidas diferentes, um `Entregue` e um `Cancelado`, e exiba o valor total de cada um.

EXERCÍCIO 2

Exercício 2 – Lojas de Hyrule

Link está percorrendo as lojas de Hyrule e precisa calcular quanto vai gastar nos itens que quer comprar. Alguns itens custam mais caro e podem garantir um desconto no total.

- Defina o tipo `Item` com os campos `nome`, `categoria` e `preco`.
- Crie o tipo `Compra` com uma lista de itens e um `StatusCompra` com os construtores `Pendente`, `Concluida` e `Cancelada`.
- Implemente `totalItens :: [Item] -> Double` que soma os preços usando `map` e `sum`.
- Implemente `valorFinal :: Compra -> Double` usando `guards`: retorne `0.0` se `Cancelada`, aplique `10%` de desconto se o total passar de `200`, e retorne o total simples nos demais casos.
- Na `main`, crie uma compra com pelo menos três itens e exiba o valor final.

EXERCÍCIO 3

Exercício 3 – Casa de Shows

Uma casa de shows precisa calcular quanto vai custar cada evento da noite. Bandas diferentes cobram cachês diferentes, e o evento pode estar ativo, encerrado ou cancelado.

- Defina o tipo Banda com os campos nome, genero e cache.
- Crie o tipo StatusEvento com os construtores Ativo, Encerrado e Cancelado.
- Crie o tipo Evento com uma lista de bandas e um StatusEvento.
- Implemente `custoTotalEvento :: Evento -> Double` usando guards: retorne 0.0 se Cancelado, e nos demais casos some todos os cachês e adicione 20% de taxa de produção.
- Implemente `bandaAbertura :: Evento -> String` e `bandaEncerramento :: Evento -> String` usando pattern matching – a abertura pega o primeiro elemento da lista e o encerramento pega o último com `last`. Ambas devem retornar uma mensagem caso a lista esteja vazia.
- Na main, crie três eventos – um Ativo, um Encerrado e um Cancelado – e exiba o custo e as bandas de abertura e encerramento de cada um.

EXERCÍCIO 4

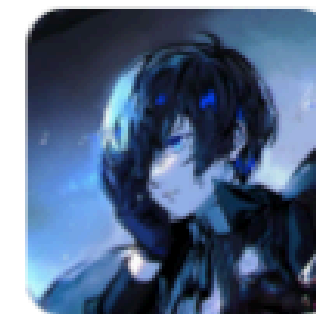
Exercício 4 – Casa de Banhos da Yubaba

Na Casa de Banhos do mundo espiritual, Yubaba registra os serviços prestados a cada criatura que chega. Cada atendimento pode ter vários serviços, e o valor final depende do que foi contratado.

- Defina o tipo `Servico` com os campos `nome`, `tipo` e `preco`.
- Crie o tipo `StatusAtendimento` com os construtores `EmAndamento`, `Finalizado` e `Cancelado`.
- Crie o tipo `Atendimento` com uma lista de serviços e um `StatusAtendimento`.
- Implemente `totalServicos :: [Servico] -> Double` que soma os preços usando `map` e `sum`.
- Implemente `valorFinalAtendimento :: Atendimento -> Double` usando `guards`: retorne `0.0` se `Cancelado`, aplique 25% de acréscimo se houver mais de 3 serviços, e retorne o total simples nos demais casos.
- Implemente `primeiroServico :: Atendimento -> String` usando `pattern matching` – retorne o nome do primeiro serviço ou uma mensagem caso a lista esteja vazia.
- Na `main`, crie dois atendimentos com serviços variados e exiba o valor final e o primeiro serviço de cada um.

FIM DA AULA!

zSh3rl0cK/S01-monitoria



 1
Contributor

 0
Issues

 0
Stars

 0
Forks



zSh3rl0cK/S01-monitoria

Contribute to zSh3rl0cK/S01-monitoria development by creating an account on GitHub.



GitHub